Milestone 3: Secure Code Review and Security Testing

Omar Soto Palos

Grand Canyon University

SWE-550: Software Engineering and Security Principles

Professor Shane Carroll

October 21, 2026

Secure Code Review

The purpose of this code review was to identify security risks in the Flask application and verify that appropriate safeguards were implemented. I reviewed the authentication process, database queries, access permissions, input validation, session management, and error handling.

#	File / Location	Security Finding	Risk Addressed	Action / Result
1	models.py create_user()	Passwords are hashed using Werkzeug before storage.	Exposure of plaintext passwords	Confirmed password hashing.
2	models.py Database queries	User-supplied values use SQL placeholders (?).	SQL injection	Confirmed parameterized queries.
3	app.py role_required()	Current user roles are verified against SQLite.	Unauthorized access and privilege escalation	Improved authorization; access tests passed.
4	app.py manage_records()	Route IDs, zones, defect counts, and dates are validated on the server.	Invalid or manipulated data	Invalid defect counts were rejected.
5	app.py and HTML templates	POST forms use Flask-WTF CSRF tokens.	Cross-site request forgery	Added protection; missing token returned HTTP 400.
6	app.py login()	Flask-Limiter restricts login submissions.	Automated password guessing	Five requests per 15 minutes per IP; HTTP 429 verified.
7	app.py Session and error handlers	Sessions time out; user-facing errors are generic.	Session reuse and information disclosure	15-minute timeout; safe 404 and 500 responses.

Security Requirements Traceability

Requirement	Implementation	Verification
SR-01 Authentication	Flask login route and SQLite credential verification	Valid login and logout tested
SR-02 Password Storage	Werkzeug password hashing in models.py	Verified through code review

Requirement	Implementation	Verification
SR-03 Role Based Access	login_required() and role_required()	Manager and team-member permissions tested
SR-04 Input Validation	Server-side checks in manage_records()	Negative defect count rejected
SR-06 Record Authorization	Manager/admin authorization on record creation	Unauthorized team-member access denied
SR-07 Login Restrictions	Flask-Limiter on /login	HTTP 429 response verified
SR-08 Session Expiration	15-minute inactivity check	Inactivity timeout reported as tested
SR-09 Error Handling	Custom 403, 404, and 500 handlers	Generic error pages verified

STRIDE Threat Mitigation

STRIDE Category	Threat	Security Control
Spoofing	An attacker attempts to gain access using guessed credentials.	Password verification, login rate limiting, and authenticated sessions
Tampering	An unauthorized user attempts to create delivery records or submit invalid information.	Backend role verification, input validation, database constraints, and CSRF protection
Information Disclosure	An application error exposes internal technical information.	Generic user-facing error messages and disabled Flask debug mode

These controls help reduce identified risks. They do not eliminate every possible attack, but they provide multiple layers of protection within the application.

Secure Code Review Summary

The secure code review confirmed that the application uses several important defensive programming practices, including password hashing, parameterized SQL queries, backend authorization, and server-side input validation.

The review also identified areas that needed improvement. CSRF protection, login rate limiting, and automatic session expiration were added to strengthen the application.

Testing demonstrated that unauthorized users were denied access to restricted pages, invalid delivery data was rejected, missing CSRF tokens caused HTTP 400 responses, and excessive login attempts produced HTTP 429 responses.

The application is a working local development prototype. Additional production security measures would include HTTPS, persistent rate-limit storage, audit logging, and broader automated security testing.

Personal Reflection

Working on this milestone helped me understand why security needs to be included throughout the software development process instead of being added at the end. When I first started building my dashboard, I was mainly focused on getting the login page, database, and delivery records to work correctly. However, reviewing the code showed me that having a working application does not necessarily mean it is secure.

One important lesson I learned was the difference between restricting access on the frontend and verifying permissions on the backend. For example, a team member might not see the option to add delivery records, but the backend still needs to prevent that user from accessing the page directly. Testing these permissions helped me understand the importance of role-based access control.

I also learned how input validation, CSRF protection, and login rate limiting help protect an application from different types of attacks. Testing these controls gave me a better understanding of how they work in practice.

Overall, this milestone helped me become more comfortable working with Python, Flask, and SQLite. It also showed me how secure coding practices can prevent problems before an application is deployed.